

PYTHON BACKEND WEB DASTURLASH KURSI

AMALIY KO'NIKMALAR IMTIHONI

Topshiriq	Maks. ball	Olingan ball
1-topshiriq	8	
2-topshiriq	10	
3-topshiriq	10	
4-topshiriq	10	
5-topshiriq (Loyiha)	12	
JAMI	50	

Ism Familiya:	
Sana:	
Guruh:	

Vaqt: 2 soat | Jami: 50 ball

Umumiy ko'rsatmalar:

- Barcha topshiriqlarga javob berishga harakat qiling.
- Kodni to'g'ri ishlashi baholashda asosiy mezon hisoblanadi.
- Funksiya nomlari va imzo (signature) topshiriqda ko'rsatilganidek bo'lsin.
- Qo'shimcha funksiya yoki kod yozish mumkin, lekin asosiy funksiya shart.
- 5-topshiriq (Django) uchun faqat kod yozing — ishga tushirish talab qilinmaydi.

1-TOPSHIRIQ Eng kichik va eng katta son*Oson*
8 BALL**Python asoslari — ro'yxatlar**

Berilgan sonlar ro'yxatidan eng kichik va eng katta sonni toping.

Natijani (min, max) ko'rinishidagi tuple sifatida qaytaring.

MUHIM: min() va max() funksiyalaridan foydalanmang!

```
def find_min_max(numbers):  
    # shu yerni to'ldiring  
    pass  
  
# Misol:  
# find_min_max([3, 7, 1, 9, 4]) --> (1, 9)  
# find_min_max([10, 5, 8, 2]) --> (2, 10)  
# find_min_max([-3, 0, 5, -1]) --> (-3, 5)
```

Baholash mezonlari:

- Eng kichik son to'g'ri — 3 ball
- Eng katta son to'g'ri — 3 ball
- min()/max() ishlatilmagan — 2 ball

Yechimingiz:**2-TOPSHIRIQ Palindrom tekshirish***Oson–O'rta*
10 BALL**Satrlar bilan ishlash**

Berilgan matn palindrom (teskari o'qilsa ham bir xil) ekanligini tekshiring.

Katta-kichik harf farqi va bo'sh joylarga e'tibor bermang.

```
def is_palindrome(text):  
    # shu yerni to'ldiring  
    pass  
  
# Misol:  
# is_palindrome("racecar") --> True  
# is_palindrome("A man a plan a canal Panama") --> True
```

```
# is_palindrome("hello") --> False  
# is_palindrome("Was it a car or a cat I saw") --> True
```

Baholash mezonlari:

- Oddiy palindromlar to'g'ri — 4 ball
- Katta-kichik harf va bo'shliq hisobga olingan — 4 ball
- Kod tozaligi va mantiq — 2 ball

Yechimingiz:

3-TOPSHIRIQ Sonlar statistikasi*O'rta*
10 BALL**Funksiyalar — hisob-kitob**

Berilgan sonlar ro'yxati uchun statistik ma'lumotlarni hisoblang.

Quyidagi kalitlardan iborat lug'at qaytaring:

```
def number_stats(numbers):  
    # shu yerni to'ldiring  
    # Qaytariladigan lug'at:  
    # {  
    #     "yigindi": barcha sonlar yig'indisi,  
    #     "ortacha": o'rtacha qiymat (float),  
    #     "juft": juft sonlar soni,  
    #     "toq": toq sonlar soni,  
    #     "musbat": musbat sonlar soni (0 dan katta),  
    #     "manfiy": manfiy sonlar soni (0 dan kichik)  
    # }  
    pass  
  
# Misol:  
# number_stats([1, 2, 3, 4, 5, 6])  
# --> {"yigindi": 21, "ortacha": 3.5, "juft": 3, "toq": 3, "musbat": 6,  
#      "manfiy": 0}  
  
# number_stats([-2, -1, 0, 1, 2])  
# --> {"yigindi": 0, "ortacha": 0.0, "juft": 4, "toq": 1, "musbat": 2,  
#      "manfiy": 2}
```

Baholash mezonlari:

- Yig'indi va o'rtacha to'g'ri — 3 ball
- Juft/toq to'g'ri — 3 ball
- Musbat/manfiy to'g'ri — 3 ball
- Bo'sh ro'yxat holatini to'g'ri hal qilsa — 1 qo'shimcha ball

Yechimingiz:

4-TOPSHIRIQ Sezar shifri*O'rta*
10 BALL**Satr va alifbo bilan ishlash**

Sezar shifri — har bir lotin harfini alifboda shift pozitsiya oldinga siljiting.

Boshqa belgilar (raqam, tinish belgilari, bo'sh joy) o'zgarmassin.

Katta harflar katta, kichik harflar kichik bo'lib qolsin.

```
def caesar_cipher(text, shift):
    # shu yerni to'ldiring
    pass

# Misol:
# caesar_cipher("hello", 3)    --> "khoor"
# caesar_cipher("xyz", 2)      --> "zab"          # alifbo davra qiladi
# caesar_cipher("Hello!", 1)   --> "Ifmmp!"       # ! o'zgarmaydi
# caesar_cipher("abc", 0)      --> "abc"          # shift=0 o'zgarmaydi
# caesar_cipher("Az", 1)       --> "Ba"          # katta A -> B, kichik z -> a
```

Baholash mezonlari:

- Kichik harflar to'g'ri siljitilsa — 3 ball
- Katta harflar to'g'ri siljitilsa — 3 ball
- Alifbo oxirida davra qilsa ($z \rightarrow a$) — 2 ball
- Boshqa belgilar o'zgarmas qolsa — 2 ball

Yechimingiz:

5-TOPSHIRIQ Django — Note (Eslatma) ilovasi*Loyiha*
12 BALL**Django MVT — Model · View · Template**

Quyida ko'rsatilgan tuzilmaga ega Django ilovasini yarating.

Har bir qismni alohida blokda yozing (models.py, views.py va h.k.).

Talab qilinadigan tuzilma:

```
# models.py
from django.db import models

class Note(models.Model):
    title = models.CharField(max_length=200)
    content = models.TextField()
    created_at = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return self.title

# Keyin: python manage.py makemigrations && python manage.py migrate
```

Yaratish kerak bo'lgan qismlar:

1. views.py — note_list(request): barcha eslatmalarni kontekst orqali shablonga jo'natadi
2. views.py — note_create(request): GET da bo'sh forma, POST da saqlash va note_list ga redirect
3. urls.py — /notes/ → note_list va /notes/create/ → note_create
4. templates/notes/list.html — eslatmalar ro'yxati + 'Yangi eslatma' havolasi
5. templates/notes/create.html — forma (title, content maydoni) + Saqlash tugmasi

Baholash mezonlari:

- Model to'g'ri yozilgan (3 maydon + __str__) — **3 ball**
- note_list view va URL ishlaydi — **2 ball**
- note_create view — GET (forma) + POST (saqlash + redirect) — **3 ball**
- Template — ro'yxat to'g'ri ko'rsatiladi — **2 ball**
- Template — forma to'g'ri ishlaydi — **2 ball**

Yechimingiz (models.py, views.py, urls.py, templates):
